



Module: Ingénierie des Systèmes d'exploitation



02 MANIPULER Windows PowerShell & Linux Shell

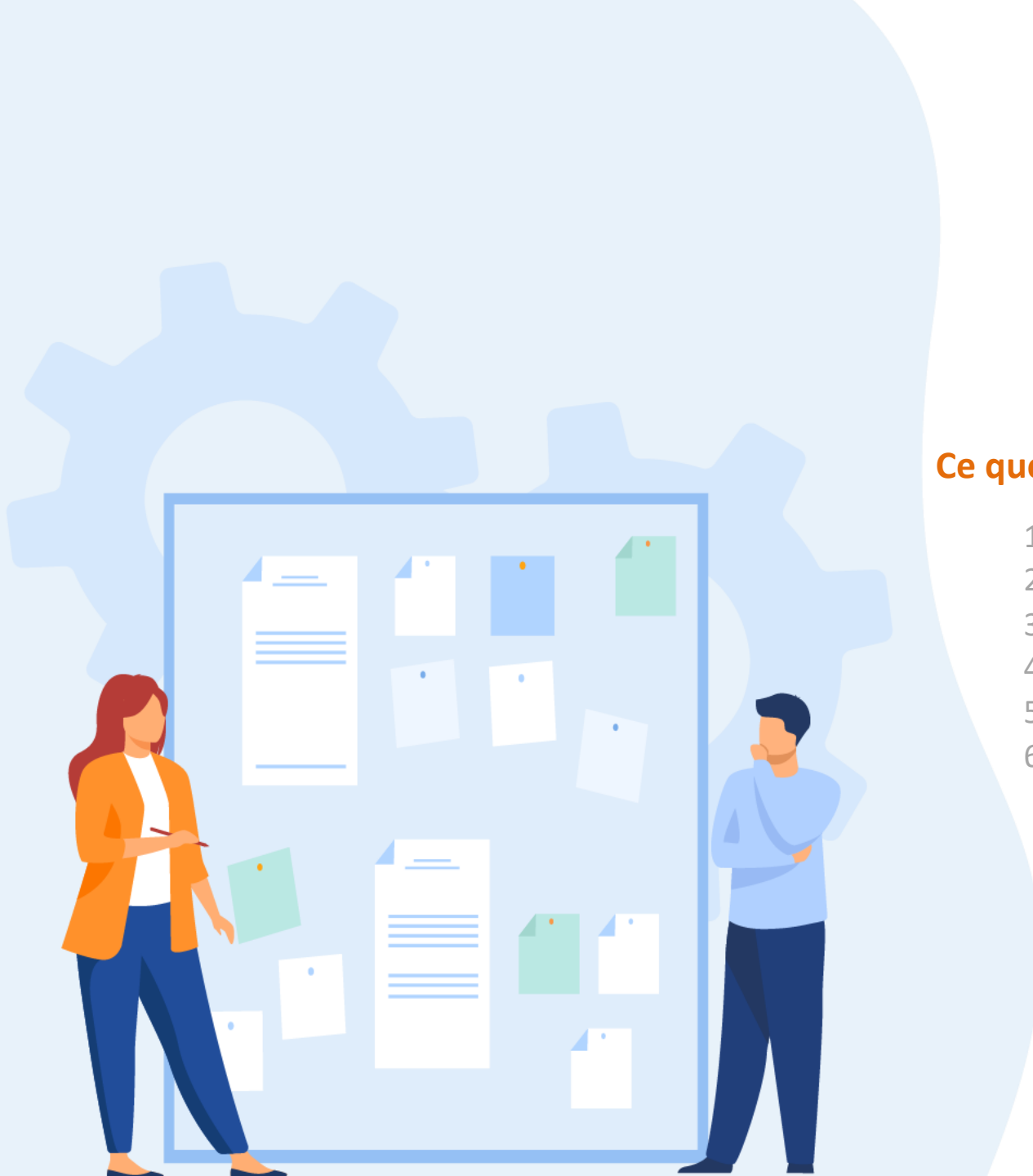
- Découvrir le « PowerShell »
- Découvrir le « SHELL »
- Maîtriser l'utilisation de PowerShell
- Maîtriser la programmation Shell

CHAPITRE 2

MANIPULER Windows PowerShell & Linux Shell

Ce que vous allez apprendre dans ce chapitre :

1. Découvrir le « PowerShell »
2. Fonctionnement Windows PowerShell
3. Découvrir le « SHELL UNIX »
4. Les commandes de base
5. les scripts shells
6. Présenter les outils d'installation de package et applications

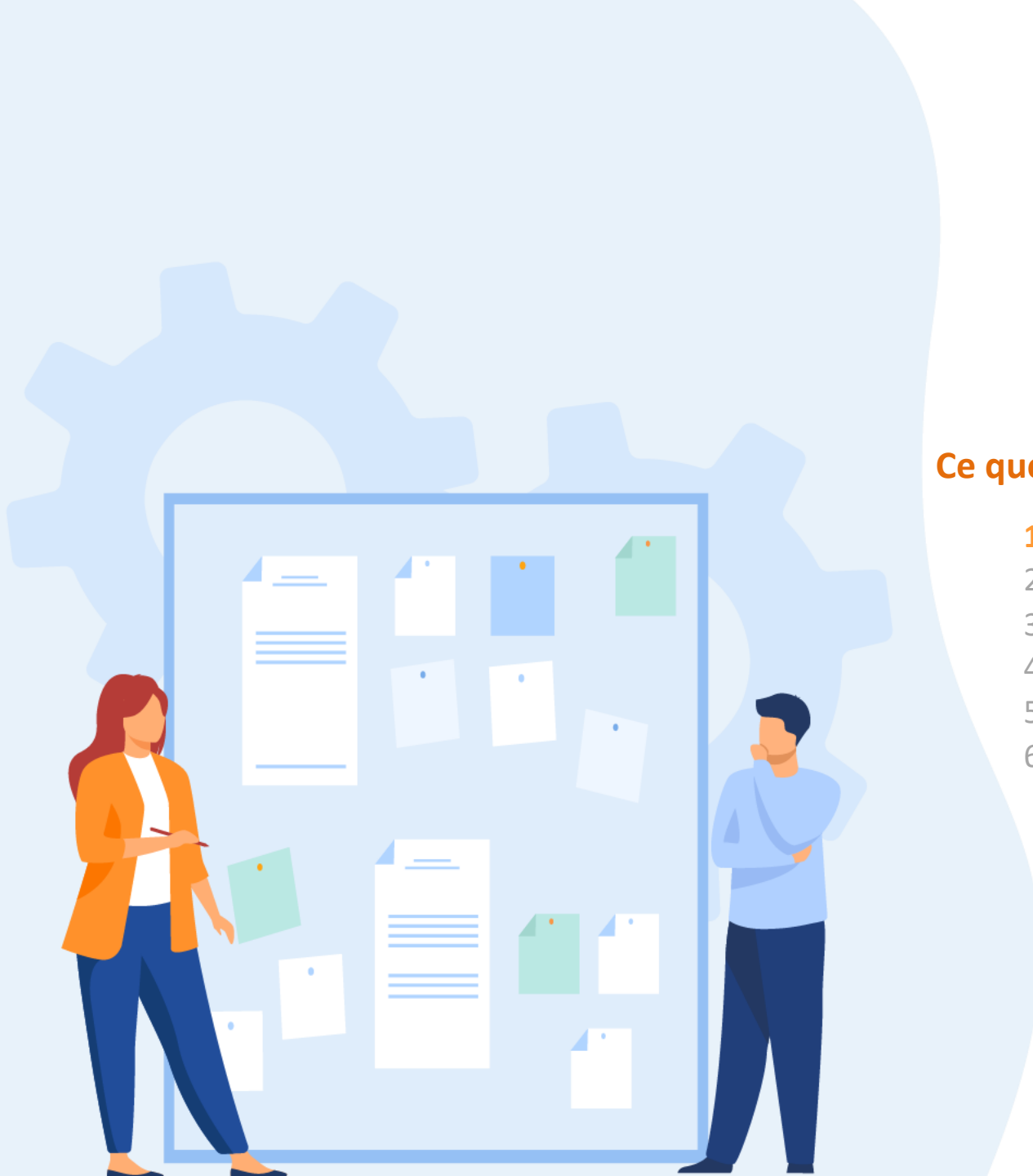


CHAPITRE 2

MANIPULER Windows PowerShell & Linux Shell

Ce que vous allez apprendre dans ce chapitre :

1. Découvrir le « PowerShell »
2. Fonctionnement Windows PowerShell
3. Découvrir le « SHELL UNIX »
4. Les commandes de base
5. les scripts shells
6. Présenter les outils d'installation de package et applications



02 – Explorer Windows

Découvrir le « PowerShell »

Windows PowerShell

En Informatique, «un **Shell**» désigne une **interface utilisateur visible** qui permet aux utilisateurs d'interagir avec les fonctions internes du système de leurs ordinateurs.

Les shells permettent aussi l'accès à une variation des fonctions et des composants profonds d'un ordinateur, ce qui les rend préférés par les professionnels de l'informatique et par les administrateurs système.

Généralement, l'**invite de commande cmd.exe** constitue le shell par défaut du systèmes Windows. Il peut être utilisé par les utilisateurs d'ordinateurs avancés pour résoudre des problèmes, ouvrir des applications de console, ou naviguer sur les lecteurs d'un PC.

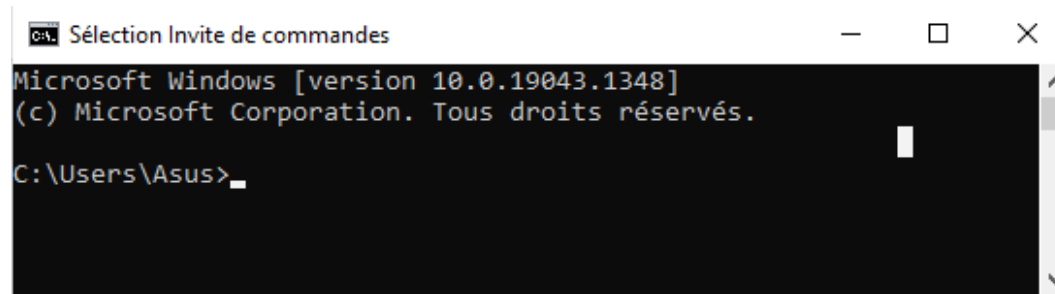


Figure : Application invite de commandes

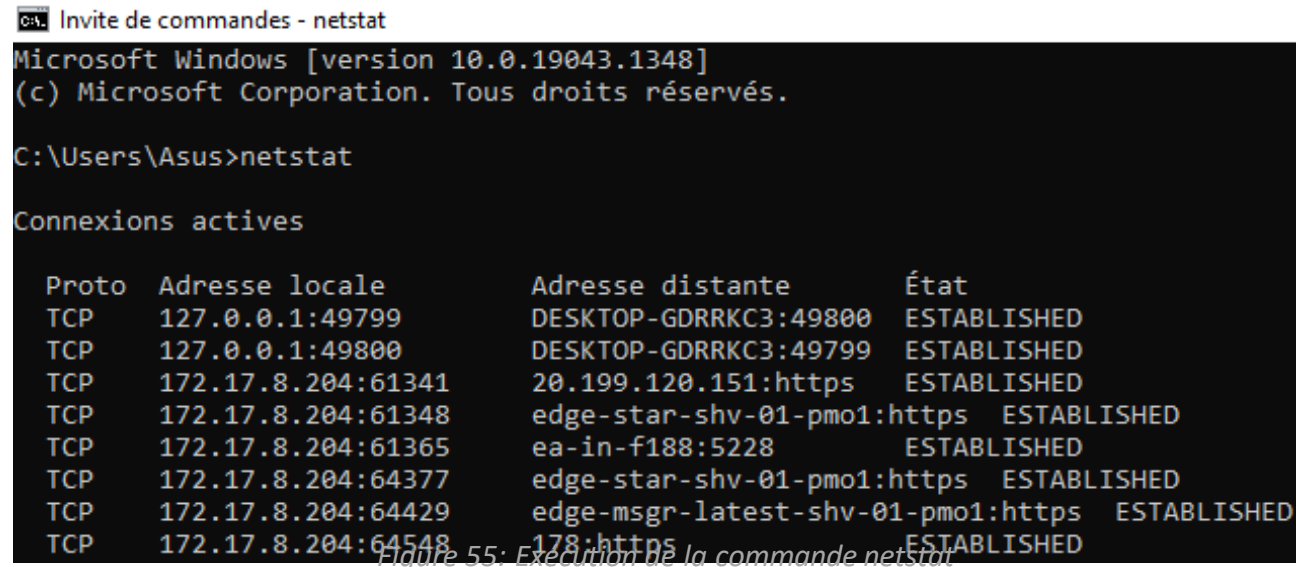
02 – Explorer Windows

Découvrir le « PowerShell »

A titre d'exemple, la commande [netstat](#) (voir Figure) peut être utilisée pour obtenir et afficher des informations de base sur toutes les activités réseau.

Néanmoins, *cmd.exe* a toujours rencontré deux problèmes : il ne permet pas l'accès à tous les composants système et présente une limite dans son langage script.

En 2008, **PowerShell** a été livré comme option avec le système d'exploitation Windows 8, et comme standard avec les versions ultérieures et même comme framework open source. Et ce afin de résoudre les limitations de *cmd.exe*



```
Microsoft Windows [version 10.0.19043.1348]
(c) Microsoft Corporation. Tous droits réservés.

C:\Users\Asus>netstat

Connexions actives

Proto  Adresse locale      Adresse distante     État
TCP    127.0.0.1:49799      DESKTOP-GDRRKC3:49800 ESTABLISHED
TCP    127.0.0.1:49800      DESKTOP-GDRRKC3:49799 ESTABLISHED
TCP    172.17.8.204:61341   20.199.120.151:https  ESTABLISHED
TCP    172.17.8.204:61348   edge-star-shv-01-pmo1:https ESTABLISHED
TCP    172.17.8.204:61365   ea-in-f188:5228       ESTABLISHED
TCP    172.17.8.204:64377   edge-star-shv-01-pmo1:https ESTABLISHED
TCP    172.17.8.204:64429   edge-msgr-latest-shv-01-pmo1:https ESTABLISHED
TCP    172.17.8.204:64548   178:https             ESTABLISHED
```

Figure 55: Exécution de la commande netstat

02 – Explorer Windows

Découvrir le « PowerShell »

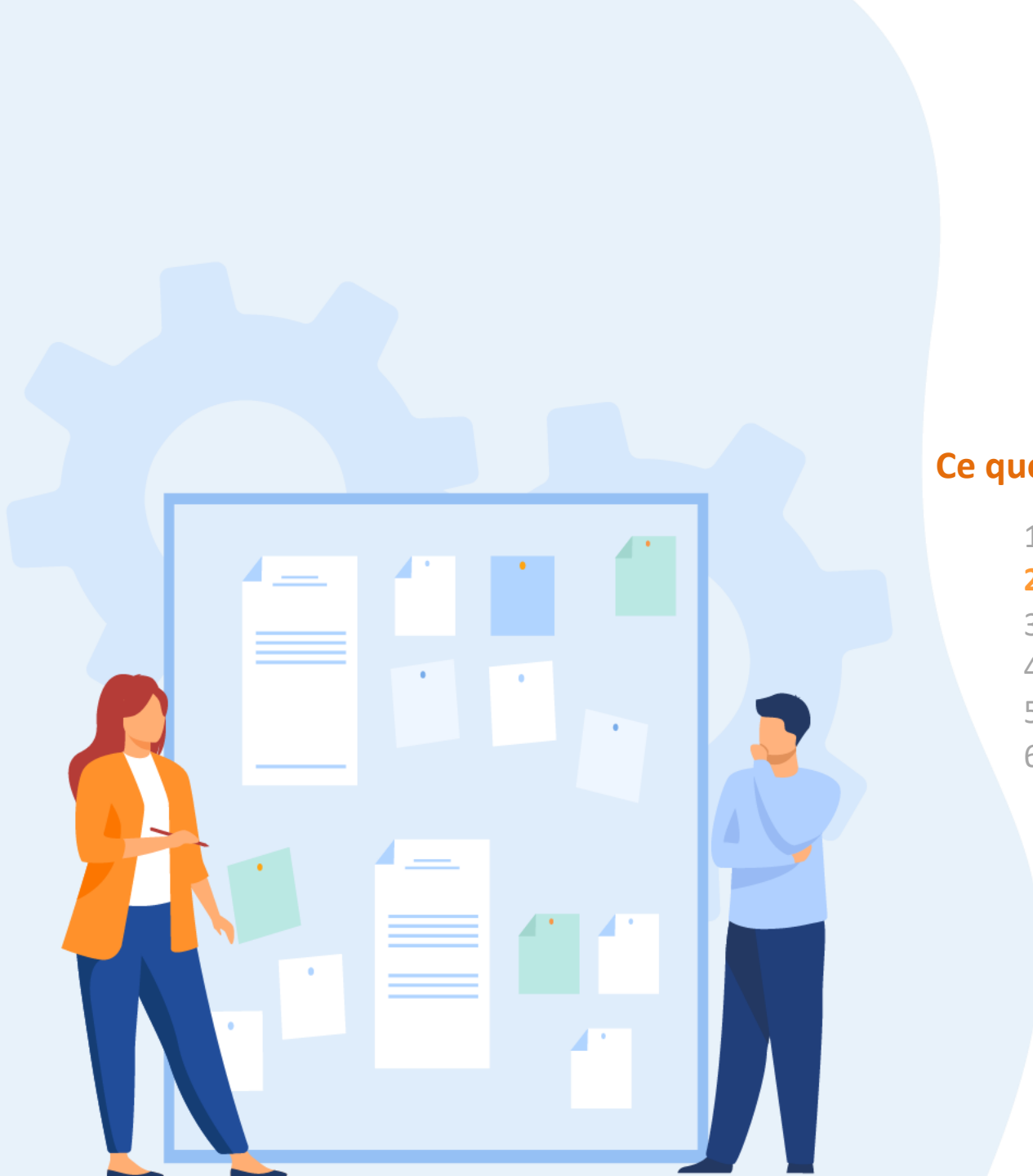
- **PowerShell** est à la fois un nouveau shell de ligne de commande et un langage de script. Il a été conçu comme moteur de tâches qui utilise des applets de commande pour encapsuler des tâches que les utilisateurs doivent effectuer.
- Dans PowerShell, vous pouvez exécuter des commandes sur des machines locales ou distantes. Vous pouvez effectuer des tâches comme gérer des utilisateurs et automatiser des workflows.

CHAPITRE 2

MANIPULER Windows PowerShell & Linux Shell

Ce que vous allez apprendre dans ce chapitre :

1. Découvrir le « PowerShell »
2. **Fonctionnement Windows PowerShell**
3. Découvrir le « SHELL UNIX »
4. Les commandes de base
5. les scripts shells
6. Présenter les outils d'installation de package et applications



02 – Explorer Windows

Fonctionnement Windows PowerShell

La Figure 1 illustre la recherche de l'application Windows PowerShell Sous Windows 10. La Figure2 présente l'interface de PowerShell.

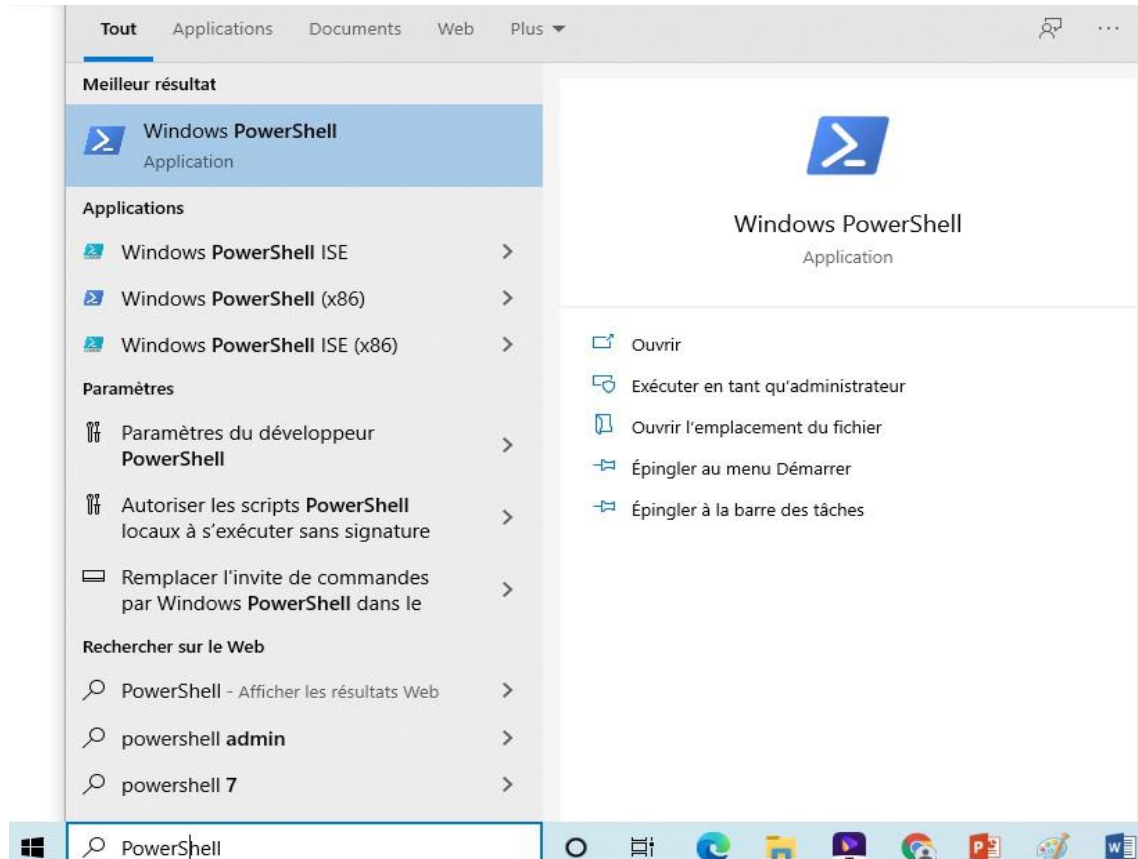


Figure 1: recherche application PowerShell sous Windows 10

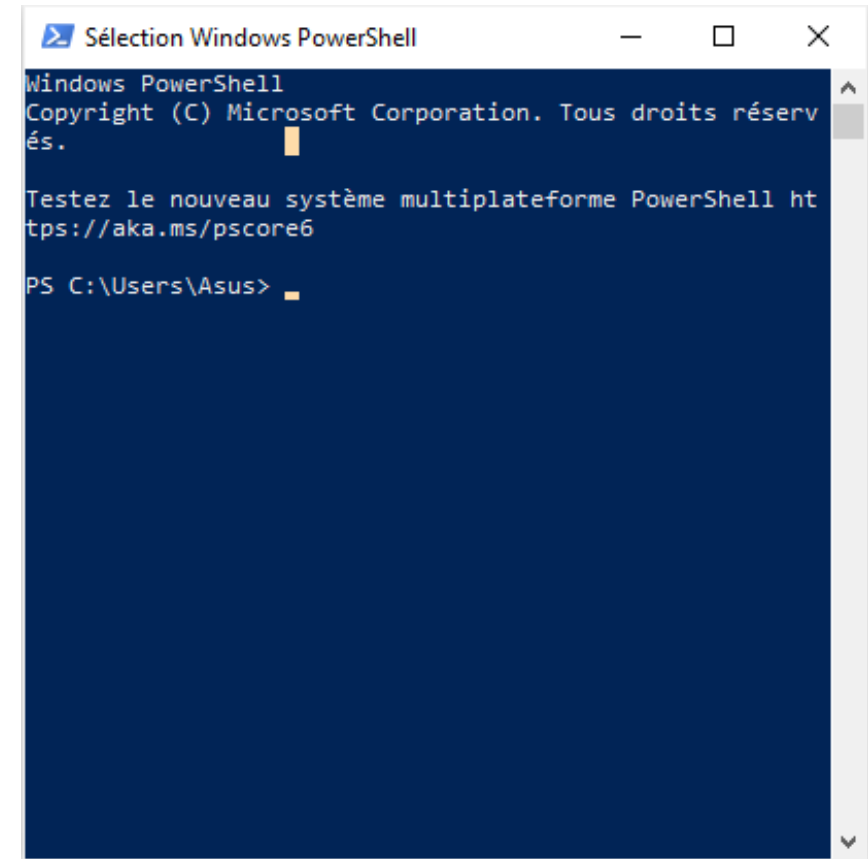


Figure 2 : Interface de l'application PowerShell sous Windows 10

02 – Explorer Windows

Fonctionnement Windows PowerShell

Fonctionnement Windows PowerShell :

PowerShell est composé principalement de deux parties, le **moteur PowerShell** et le **langage de script PowerShell**. On peut utiliser les deux soit séparément afin de tirer le meilleur parti du programme.

Le moteur PowerShell

L’**interpréteur de ligne de commande** appelé en anglais *command-line interpreter*, abrégé par CLI de PowerShell, permet essentiellement à l’utilisateur **d’accéder aux différentes fonctions internes du système d’exploitation SE**. Les commandes du programme sont nommées ***cmdlets*** (*commandlets*, signifie *petites commandes*). Ces commandes sont toujours composées **d’un verbe et d’un nom au singulier**, à titre d’exemple, *Stop-Process* ou *Sort-Object*.

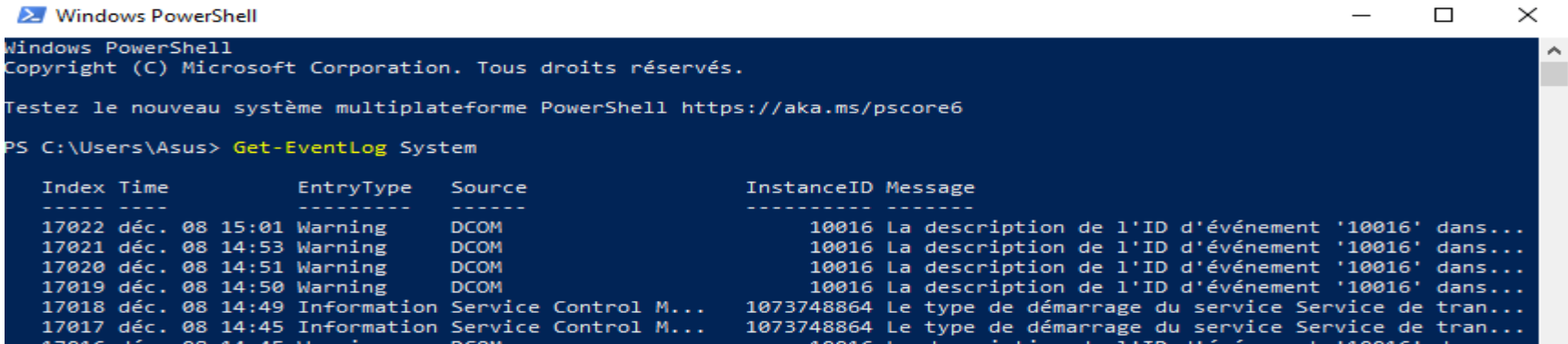


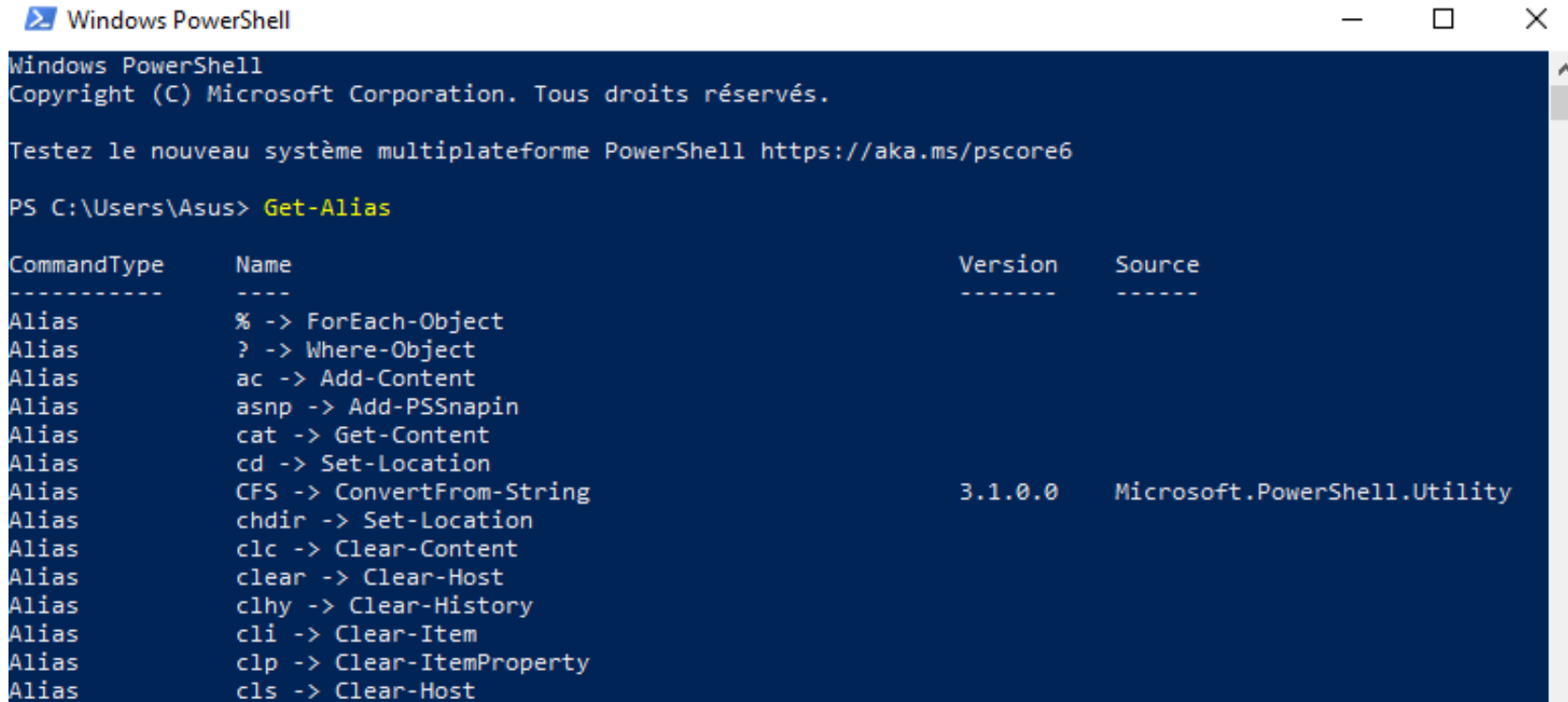
Figure : Exécution de la commande Get-EventLog

02 – Explorer Windows

Fonctionnement Windows PowerShell

Shell offre plus de 100 **commandlets** de base qui contiennent des commandes standard couramment utilisées à partir de l'invite des commandes et qui sont disponibles sous la forme d'un alias.

Pour afficher la liste des commandes, vous pouvez utiliser la commande suivante : ***Get-Alias***.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Testez le nouveau système multiplateforme PowerShell https://aka.ms/pscore6

PS C:\Users\Asus> Get-Alias

CommandType      Name                                Version      Source
-----
Alias             % -> ForEach-Object
Alias             ? -> Where-Object
Alias             ac -> Add-Content
Alias             asnp -> Add-PSSnapin
Alias             cat -> Get-Content
Alias             cd -> Set-Location
Alias             CFS -> ConvertFrom-String          3.1.0.0     Microsoft.PowerShell.Utility
Alias             chdir -> Set-Location
Alias             clc -> Clear-Content
Alias             clear -> Clear-Host
Alias             clhy -> Clear-History
Alias             cli -> Clear-Item
Alias             clp -> Clear-ItemProperty
Alias             cls -> Clear-Host
```

Figure : Avec la commande *Get-Alias* tous les alias par défaut de PowerShell sont affichés.

02 – Explorer Windows

Fonctionnement Windows PowerShell

Cette table contient des exemples de cmdlets fréquemment utilisées ainsi que leurs alias associés :

Table : Exemple de commande Shell.

Alias	Cmdlet	Fonction
Cd	Set-Location	Modifier le répertoire courant
dir	Get-ChildItem	Lister tous les éléments d'un dossier
gi	Get-Item	Appeler un élément spécifique
Ps	Get-Process	Lister tous les processus
gsv	Get-Service	Lister tous les services installés
gm	Get-Member	Afficher toutes les propriétés et méthodes d'un objet
clear	Clear-Host	Vider l'hôte PowerShell

02 – Explorer Windows

Fonctionnement Windows PowerShell

Une grande partie du [langage de script](#) de PowerShell peut vous sembler familier à partir de **C#**. Non seulement vous pouvez l'utiliser pour écrire vos propres cmdlets (et les partager avec d'autres utilisateurs si nécessaire), mais vous pouvez **également regrouper plusieurs commandes consécutives dans un fichier script.ps1** pour **étendre les fonctionnalités du shell**.

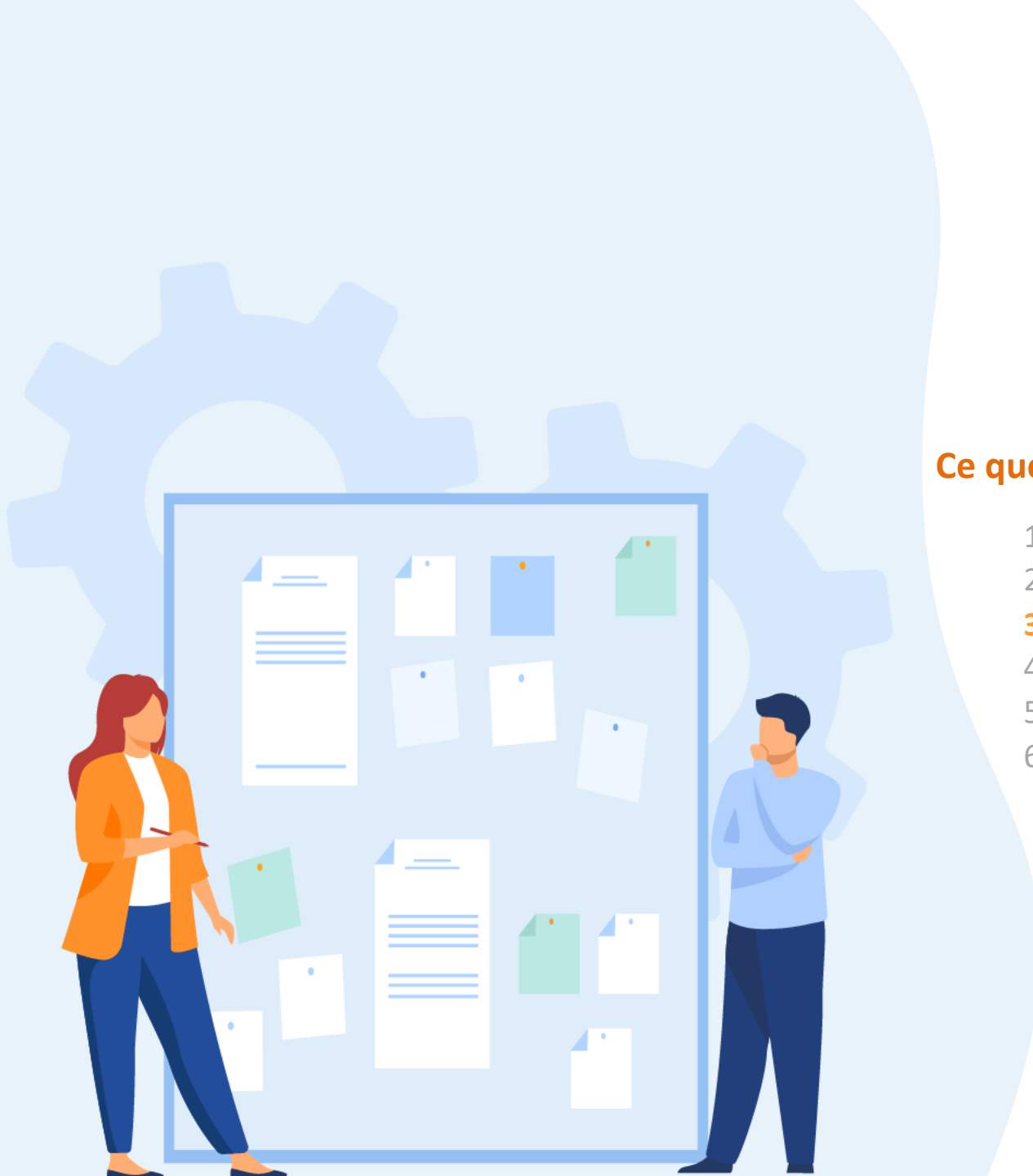
Les scripts contiennent toujours une description de l'application et sont exécutés avec le préfixe « . » suivi du chemin complet du fichier.

CHAPITRE 2

MANIPULER Windows PowerShell & Linux Shell

Ce que vous allez apprendre dans ce chapitre :

1. Découvrir le « PowerShell »
2. Fonctionnement Windows PowerShell
3. **Découvrir le « SHELL »**
4. Les Commandes de base
5. les scripts shells
6. Présenter les outils d'installation de package et applications



02 – Manipuler le Shell Linux

Découvrir le « SHELL »

Linux fournit un Shell et permet la programmation Shell .

Définition d'un shell :

Un shell assure deux rôles principaux :

- 1.C'est un interpréteur de commandes ;
- 2.C'est un langage de programmation.

```
[root@localhost ~]# ls -l /bin/bash
-rwxr-xr-x. 1 root root 960392  2 août   2016 /bin/bash
[root@localhost ~]# ls -l /bin/sh
lrwxrwxrwx. 1 root root 4 16 févr. 12:28 /bin/sh -> bash
[root@localhost ~]#
```

```
sahar@ubuntu:~$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1021112 May 16   2017 /bin/bash
sahar@ubuntu:~$ ls -l /bin/sh
lrwxrwxrwx 1 root root 4 Sep 18 08:18 /bin/sh -> dash
sahar@ubuntu:~$
```

Figure : Terminal Linux Utilisation commande Shell (sh)

02 – Manipuler le Shell Linux

Découvrir le « SHELL »

Modes d'utilisation d'un shell :

1. Interactif : l'utilisateur saisit et exécute ses lignes de commandes une par une dans un terminal ;
2. Non interactif : le shell lit un ensemble de commandes à partir d'un fichier appelé shell script.

```
[root@localhost ~]# ls -l /bin/bash
-rwxr-xr-x. 1 root root 960392  2 août  2016 /bin/bash
[root@localhost ~]# ls -l /bin/sh
lrwxrwxrwx. 1 root root 4 16 févr. 12:28 /bin/sh -> bash
[root@localhost ~]#
```

```
sahar@ubuntu:~$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1021112 May 16  2017 /bin/bash
sahar@ubuntu:~$ ls -l /bin/sh
lrwxrwxrwx 1 root root 4 Sep 18 08:18 /bin/sh -> dash
sahar@ubuntu:~$
```

Figure : Terminal Linux Utilisation commande Shell (sh)

02 – Manipuler le Shell Linux

Découvrir le « SHELL »

Type du shell :

Les systèmes Unix utilisent différents interpréteurs de commandes (shells) :

1. Bash (Bourne Again SHell) : Est le shell par défaut sur la plupart des distributions Linux
2. SH (Bourne Shell): Assez basique mais largement compatible avec de nombreuses versions Unix
3. Bsh (Bourne Shell) : Similaire à SH et a été largement utilisé dans les premiers systèmes Unix.
4. Csh (C Shell) : Ecrit en langage C, conçu pour offrir une syntaxe plus conviviale que le Bourne Shell (SH).
5. Tcsh (Tenex C Shell) : Version améliorée de Csh

```
[root@localhost ~]# ls -l /bin/bash
-rwxr-xr-x. 1 root root 960392  2 août  2016 /bin/bash
[root@localhost ~]# ls -l /bin/sh
lrwxrwxrwx. 1 root root 4 16 févr. 12:28 /bin/sh -> bash
[root@localhost ~]#
```

```
sahar@ubuntu:~$ ls -l /bin/bash
-rwxr-xr-x 1 root root 1021112 May 16  2017 /bin/bash
sahar@ubuntu:~$ ls -l /bin/sh
lrwxrwxrwx 1 root root 4 Sep 18 08:18 /bin/sh -> dash
sahar@ubuntu:~$
```

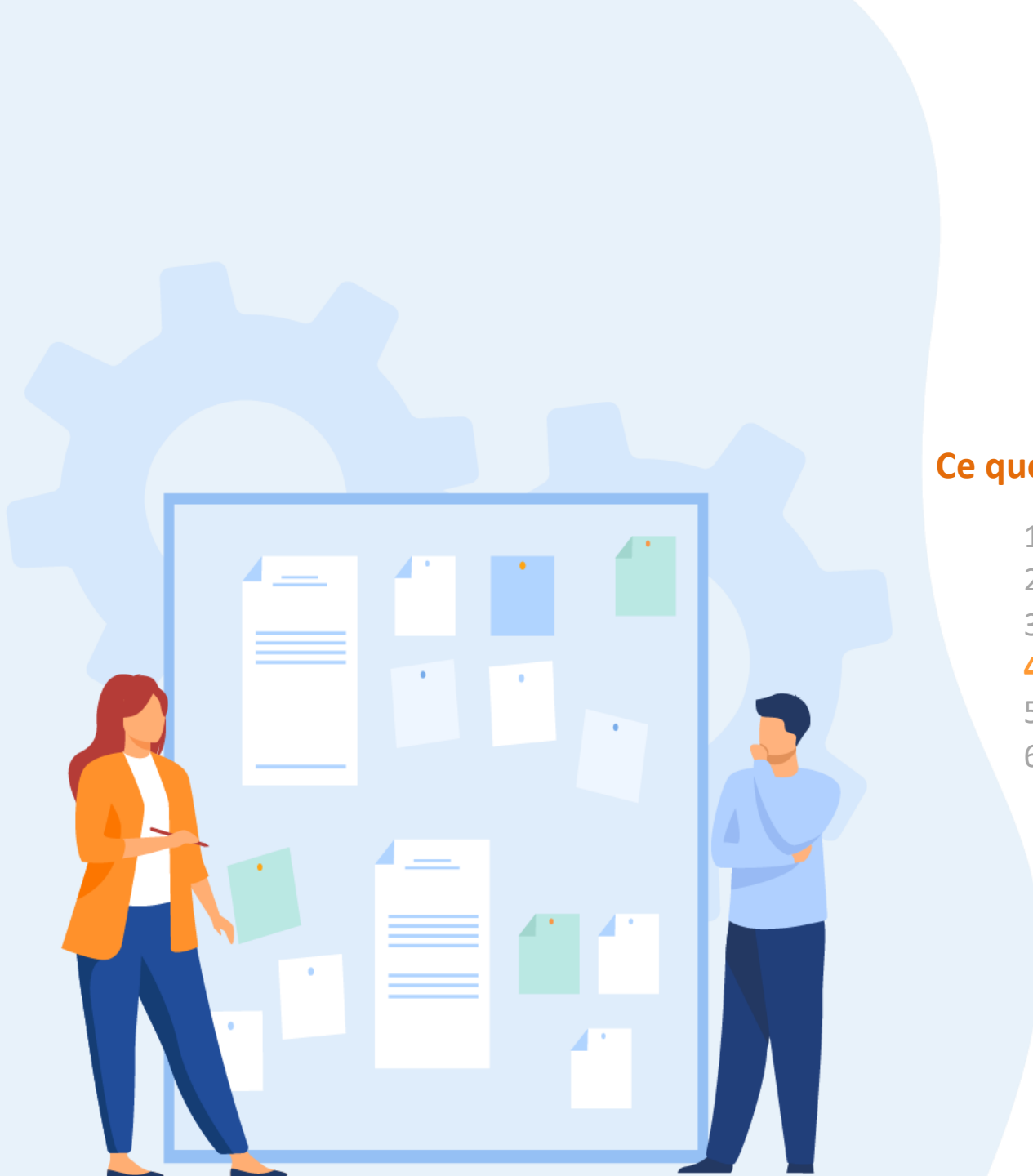
Figure : Terminal Linux Utilisation commande Shell (sh)

CHAPITRE 2

MANIPULER Windows PowerShell & Linux Shell

Ce que vous allez apprendre dans ce chapitre :

1. Découvrir le « PowerShell »
2. Fonctionnement Windows PowerShell
3. Découvrir le « SHELL »
4. **Les Commandes de base**
5. les scripts shells
6. Présenter les outils d'installation de package et applications



02 – Manipuler le Shell Linux

Les Commandes de base

Assurer la gestion de base du système de fichiers

Afin de permettre à l'utilisateur la gestion du système de fichier, Linux propose un ensemble de commandes de base permettant la manipulation des fichiers décrits dans

la Table 1, 2 et 3.

Fonctionnalité	Syntaxe
Affichage de répertoire courant	<code>pwd</code>
Copie d'un fichier	<code>cp fich-orig fich-dest</code> <code>cp fichier repertoire</code>
Copie des répertoires entiers	<code>cp -r rep-origine rep-destination</code> <code>rsync -a rep-origine/ rep-destination/</code>
Renommer fichier1 en fichier2 (idem pour répertoire)	<code>mv fichier1 fichier2</code>

Table 1 : Commande de base (1)

02 – Manipuler le Shell Linux

Les Commandes de base

Assurer la gestion de base du système de fichiers

On vous invite à tester ces commandes dans votre terminal sous linux.

Fonctionnalité	Syntaxe
Déplacer fichier1 dans le répertoire /tmp	<code>mv fichier1 /tmp</code>
Déplace le répertoire TEST dans le répertoire /tmp	<code>mv TEST /tmp</code>
Supprimer des fichiers	<code>rm fichier1 fichier2</code>
Supprimer un répertoire vide	<code>rmdir rep</code>

Table 2 : Commande de base (2)

02 – Manipuler le Shell Linux

Les Commandes de base

Assurer la gestion de base du système de fichiers

Fonctionnalité	Syntaxe
Supprimer un répertoire non vide	<code>rm -rf rep</code>
Lister des fichiers du dossier courant	<code>ls</code>
Afficher une liste détaillée	<code>ls -l</code>
Lister tous les fichiers (cachés)	<code>ls -a</code>
Trier les fichiers par date (plus récents sont les premiers)	<code>ls -t</code>
Trier les fichiers par taille	<code>ls -S</code>
Changer rep	<code>cd</code>
Créer un rep	<code>mkdir rep</code>
Crée une arborescence rep_parent: dans rep_parent , crée rep_enfant	<code>mkdir -p rep1/rep2</code>

Table 9 : Commande de base (3)

02 – Manipuler le Shell Linux

Les Commandes de base

Exercice 1

❖ Avec une seule commande et en utilisant l'expansion des accolades dans Bash, créez 5 fichiers numérotés de 1 à 5 avec le préfixe jeu (jeu1, jeu2, ...).

02 – Manipuler le Shell Linux

Les Commandes de base

Solution

```
$ touch jeu{1..5}  
$ ls  
jeu1  jeu2  jeu3  jeu4  jeu5
```

02 – Manipuler le Shell Linux

Les scripts shells

Débuter avec les scripts shells

- Un script shell est un simple fichier texte exécutable (avec un droit d'exécution x).
- Il doit impérativement commencer par une ligne indiquant au système le shell qu'il faut utiliser :

Shebang → **#!** /chemin/interpréteur
Exemple : `/bin/bash` ou `/bin/sh`

- Le script doit être rendu exécutable : `chmod +x fichier`.
- Dans un script shell on peut avoir : des variables, des structures de contrôles, des structures répétitives... d'où l'appellation **script shell**.

02 – Manipuler le Shell Linux

Les scripts shells

Exemple 1 : Premier script shell

Ecrire un script qui affiche **Bonjour ENIAD**

02 – Manipuler le Shell Linux

Les scripts shells

Exemple 1 : Premier script shell

Ecrire un script qui affiche Bonjour ENIAD

Solution :

```
#!/bin/bash  
Echo "bonjour ENIAD"
```

02 – Manipuler le Shell Linux

Les scripts shells

Les Entrées-Sorties:

Ce sont les voies de communication entre le programme bash et la console.

- **echo**: Affiche l'argument texte entre guillemets sur la sortie standard.
- **read**: Permet l'affectation directe par lecture de la valeur saisie sur l'entrée standard au clavier.

Exemple 2 :

Ecrire un script appelé script1 qui demande votre nom et prénom et affiche **Bonjour votre_nom votre_prenom**.

02 – Manipuler le Shell Linux

Les scripts shells

Solution Exemple 2 :

```
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
GNU nano 2.3.1      Fichier : fichier
#!/bin/bash
echo "Entrez votre nom : "
read nom
echo "Entrez votre prenom : "
read prenom
echo "Bonjour $nom $prenom "
```

```
Fichier  Édition  Affichage  Rechercher  Terminal  Aide
[root@localhost ~]# nano fichier
[root@localhost ~]# bash fichier
Entrez votre nom :
tekup
Entrez votre prenom :
student
Bonjour tekup student
[root@localhost ~]# █
```

02 – Manipuler le Shell Linux

Les scripts shells

Variables d'environnement:

Ces variables sont accessibles par tous les processus fils du shell courant

Parmi ces variables nous citons :

- **DISPLAY** : L'écran sur lequel les programmes X travaillent..
- **PRINTER** : pour les commandes d'impression. Contient le nom de l'imprimante sur laquelle il faut envoyer vos fichiers.
- **SHELL** : contient le nom de votre shell.
- **HOME** : contient le nom de votre répertoire personnel.
- **USER** : contient votre nom de login.
- **LOGNAME** : la même chose que USER.
- **PATH** : contient une liste de répertoires dans lesquels le shell va chercher les commandes.

02 – Manipuler le Shell Linux

Les scripts shells

Variables d'environnement (Exemple):

Exemple :

```
#!/bin/bash  
# Fichier "connexion"  
  
echo "le répertoire de connexion est : $HOME"
```

Remarque :

La commande **env** permet d'afficher l'ensemble des variables d'environnement pour le shell actif.

02 – Manipuler le Shell Linux

Les scripts shells

Variables locales:

Ces variables sont définies dans un bloc de code spécifique, tel qu'une fonction ou un script, et sont accessibles uniquement dans ce bloc.

Pour référencer la valeur d'une variable, on utilise la notation consistant à écrire le signe \$ suivi du nom de la variable.

```
i=2
```

```
echo $i
```

```
2
```

02 – Manipuler le Shell Linux

Les scripts shells

Les variables prépositionnées

Ces variables sont automatiquement affectées lors d'un appel de script suivi d'une liste de paramètres (voir le tableau ci-dessous) :

Variable	Interprétation
\$?	C'est la valeur de sortie de la dernière commande. Elle vaut 0 si la commande s'est déroulée sans problème.
\$0	Cette variable contient le nom du script
\$1 à \$9	Les (éventuels) premiers arguments passés à l'appel du script
\$#	Le nombre d'arguments passés au script
\$*	La liste des arguments à partir de \$1
\$\$	le n° PID du processus courant
\$!	le n° PID du processus fils

02 – Manipuler le Shell Linux

Les scripts shells

Les variables prépositionnées

Exemple:

```
#!/bin/bash
# Fichier "variableP.sh"

echo "Nom du script $0"
echo "premier paramètre $1"
echo "second paramètre $2"
echo "troisième paramètre $3"
echo "PID du shell " $$
echo "code de retour $?"
echo "nombre d'arguments :" $#
echo "tous les paramètres :" $*
```

```
[ofppt@localhost ~]$ ./variableP.sh 10 cisco linux
Nom du script ./variableP.sh
premier paramètre 10
second paramètre cisco
troisième paramètre linux
PID du shell 43698
code de retour 0
nombre d'arguments : 3
tous les paramètres : 10 cisco linux
```

02 – Manipuler le Shell Linux

Les scripts shells

Les opérateurs arithmétiques sont décrits dans la table ci-dessous.

Opérateur	Rôle
+	Addition
-	Soustraction
*	Multiplication
**	exponentiation : $a^b = a^{**}b$
/	Division
%	Modulo

Exemple :

```
[sahar@localhost ~]$ a=9
[sahar@localhost ~]$ b=10
[sahar@localhost ~]$ echo $((a+5))
14
[sahar@localhost ~]$ echo $((a-5))
4
[sahar@localhost ~]$ echo $((a**2))
81
```

```
[sahar@localhost ~]$ A=5
[sahar@localhost ~]$ B=6
[sahar@localhost ~]$ ((somme=A+B))
[sahar@localhost ~]$ echo $somme
11
[sahar@localhost ~]$ ((produit=A*B))
[sahar@localhost ~]$ echo $produit
30
```

02 – Manipuler le Shell Linux

Les scripts shells

Les opérateurs Logiques permettent la comparaison de nombres

<code>\$num1 -eq \$num2</code>	Vérifie que les 2 nombres sont égaux
<code>\$num1 -ne \$num2</code>	Vérifie que les 2 nombres ne sont pas égaux
<code>\$num1 -lt \$num2</code>	Vérifie si num1 est inférieur (<) à num2
<code>\$num1 -le \$num2</code>	Vérifie si num1 est inférieur ou égal (<=) à num2
<code>\$num1 -gt \$num2</code>	Vérifie si num1 est supérieur (>) à num2
<code>\$num1 -ge \$num2</code>	Vérifie si num1 est supérieur ou égal (>=) à num2

02 – Manipuler le Shell Linux

Les scripts shells

Structures conditionnelles pour les comparaison de variables contenant du texte

<code>\$texte1 = \$texte2</code>	Vérifie que la variable <code>texte1</code> contient la même chose que <code>texte2</code>
<code>\$texte1 != \$texte2</code>	Vérifie que les deux variables sont différentes
<code>-z \$texte</code>	Vérifie si la variable est vide (pas de texte) ou inexistante.
<code>-n \$texte</code>	Vérifie si la variable est non vide

02 – Manipuler le Shell Linux

Les scripts shells

Certains **opérateurs** permettent de faire des tests sur des fichiers.

<code>-e \$nomfichier</code>	Vérifie si le fichier existe
<code>-d \$nomfichier</code>	Vérifie que le fichier est un répertoire.
<code>-f \$nomfichier</code>	Vérifier que le fichier est un fichier (donc pas un répertoire)

02 – Manipuler le Shell Linux

Les scripts shells

❑ La structure de contrôle : if

L'instruction conditionnelle permet :

- de reconnaître les caractéristiques des fichiers et des répertoires ;
- de comparer des chaînes de caractères ;
- de comparer algébriquement des nombres.

```
if cmd1  
then Instructions1;  
elif cmd2  
then Instructions2;  
else Instructions4;  
fi
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ La structure de contrôle : if

► Exemples :

```
if [ -z $a ]  
then  
    echo "La variable est vide"  
fi
```

```
#  
if [ $b -eq $c ]  
then  
    echo "les valeurs sont égales"  
fi
```

```
#  
if [ $c -gt $b ]  
then  
    echo "b est supérieure à c"  
fi
```

```
#  
if [ $b -lt $c ]  
then  
    echo "b est inférieure c"  
fi
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ Opérateur ET (&&)

Il est possible d'assembler 2 conditions entre-elles pour former condition plus complexe qui n'est valide que si les deux sous-conditions sont vérifiées. C'est l'opérateur ET qui s'écrit &&.

```
if [ $prenom = "aaron" ] && [ $nom = "swartz" ]  
then  
    echo "Bonjour Aaron Swartz"  
fi
```

❑ Opérateur OU (||)

Permet de faire la même chose que && mais seulement si au moins une des conditions est correcte.

```
if [ $jour = "samedi" ] || [ $jour = "dimanche" ]  
then  
    echo "C'est le week-end"  
fi
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ La structure de contrôle : if

Exercice 2

❖ Appréciation de note :

- Créer un script qui demande à l'utilisateur de saisir une note et qui affiche un message en fonction de cette note :

"très bien" si la note est entre 16 et 20 ;

"bien" lorsqu'elle est entre 14 et 16 ;

"assez bien" si la note est entre 12 et 14 ;

"moyen" si la note est entre 10 et 12 ;

"insuffisant" si la note est inférieur à 10.

02 – Manipuler le Shell Linux

Les scripts shells

❑ La structure de contrôle : if

solution

```
#!/bin/bash
echo "Entrez votre note : "
read -r note
if [ "$note" -ge 16 ]; then
    echo "très bien"
elif [ "$note" -ge 14 ]; then
    echo "bien"
elif [ "$note" -ge 12 ]; then
    echo "assez bien"
elif [ "$note" -ge 10 ]; then
    echo "moyen"
else
    echo "insuffisant"
fi
```

02 – Manipuler le Shell Linux

Les scripts shells

La boucle **for** est décrit dans la table ci-dessous.

	Forme 1	Forme 2	Forme 3
Syntaxe	for variable in ch1 ch2... chn do Commandes done	for variable do Commandes done	for variable in * do commandes done
Interprétation	les valeurs de variable sont les chaines de ch 1 à ch n	variable prend ses valeurs dans la liste des paramètres du script.	la liste des fichiers du répertoire constitue les valeurs prises par variable.

Itération **while** : elle correspond à l’itération telle que décrit dans la table ci-dessous.

```
while suite_cmd1  
do  
    suite_cmd2  
done
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ La Boucle FOR

Exercice 3

- Créer un script qui prend un nombre en saisie et l'élève à sa propre puissance. C'est un peu le même principe que la factorielle mais cette fois, l'usage de la boucle for est imposé.

- Exemple d'exécution :

[~] ./NomDuScript.sh

Saisir une valeur :

2

$2^2 = 4$

02 – Manipuler le Shell Linux

Les scripts shells

❑ La boucle FOR

Solution

```
#!/bin/bash
echo "Saisir une valeur"
read -r value
result=1
for (( i=0 ; i<$value ; i++ )); do
    let result=$result*$value
done
echo "$value^$value = $result"
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ Tâche palanifiée crontab

Il est chargé de faire exécuter par le système toutes tâches (commandes et scripts) définies et planifiées à l'avance; Ces tâches allant de la simple commande aux scripts complexes peuvent ainsi être exécutées à heure fixe et même de façon périodique, et fait l'objet de messages de compte-rendu.

Option :

- **crontab -l** affiche le fichier crontab de l'utilisateur
- **crontab -r** efface ce fichier
- **crontab -e** crée ou édite (pour modification) un fichier temporaire

02 – Manipuler le Shell Linux

Les scripts shells

❑ Tâche planifiée crontab

Un fichier crontab comporte cinq champs – chaque champ est représenté par un astérisque.

```
# _____ minute (0 - 59)
# |_____ heure (0 - 23)
# | |_____ jour du mois (1 - 31)
# | | |_____ mois (1 - 12)
# | | | |_____ jour de la semaine (0 - 7) (du dimanche au samedi ;
# | | | | |_____ 7 est aussi le dimanche sur certains systèmes)
# | | | | |
# | | | | |
# * * * * * commande à exécuter
```

Minute — minute de l'heure à laquelle la commande sera exécutée, allant de 0 à 59.

Heure — l'heure à laquelle la commande sera exécutée, de 0 à 23.

Jour du mois — le jour du mois où vous voulez que la commande soit exécutée, de 1 à 31.

Mois — le mois sur lequel la commande spécifiée sera exécutée, de 1 à 12.

Jour de la semaine — le jour de la semaine où vous souhaitez que la commande soit exécutée, de 0 à 7.

02 – Manipuler le Shell Linux

Les scripts shells

❑ Tâche planifiée crontab

Exemples

- ▶ # exécution chaque 1er et 15 de chaque mois à minuit

```
0 0 1,15 * * commande
```

- ▶ # provoquer un reboot la machine chaque 1er et 15 du mois à 2h 30 du matin

```
30 2 1,15 * * /sbin/shutdown -r
```

- ▶ # appeler un script de sauvegarde tous les lundis à 3 h 15 du matin

```
15 3 * * 1 /usr/bin/backup
```

- ▶ # exécution toutes les heures passées 15 minutes

```
15 * * * * commande
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ Routage

La commande **route** affiche la table de routage. Par exemple:

```
$ route
```

Destination	Passerelle	Genmask	Indic	Metric	Ref	Use
<u>Iface</u>						
172.17.13.0	*	255.255.255.0	U	0	0	0
eth2						
193.190.211.0	*	255.255.255.0	U	0	0	0
eth0						
192.168.13.0	*	255.255.255.0	U	0	0	0
eth2						
172.16.12.0	*	255.255.254.0	U	0	0	0
eth2						
default	193.190.211.1	0.0.0.0	UG	0	0	0
eth0						

La commande **netstat -rn** affiche la table de routage. Par exemple:

```
# netstat -rn
```

Routing Table: IPv4						
Destination	Gateway	Flags	Ref	Use	Interface	
-----	-----	-----	-----	-----	-----	
default	172.20.1.10	UG	1	249	ce0	
224.0.0.0	172.20.1.10	U	1	0	ce0	
10.0.5.0	10.0.5.20	U	1	78	bge0	
127.0.0.1	127.0.0.1	UH	1	57	lo0	

02 – Manipuler le Shell Linux

Les scripts shells

❑ Routage

Pour Ajouter une route utiliser la commande :

route -p add -net network-address -gateway gateway-address

- **-p** : Crée une route qui doit être conservée après les réinitialisations du système.
- **Add** :Indique que vous êtes sur le point d'ajouter la route
- **-net adresse-réseau** : Indique que la route intègre le réseau avec l'adresse adresse-réseau.
- **-gateway adresse-passerelle** :Indique que le système de passerelle pour la route spécifiée possède l'adresse IP adresse-passerelle

```
# route add -net 192.168.55.0 netmask 255.255.255.0 gw 192.168.0.1
$ route -n
Table de routage IP du noyau
Destination      Passerelle      Genmask      Indic Metric Ref      Use
Iface
192.168.55.0     192.168.0.1      255.255.255.0  UG      0      0      0
eth1
..
```

02 – Manipuler le Shell Linux

Les scripts shells

❑ Routage

Pour ajouter une route par défaut en passant on utilise la commande suivante:

#route add default gw Adresse IP de la passerelle Nom de la carte réseau

Exemple:

Pour ajouter une route par défaut en passant par la passerelle 192.168.0.1; on peut utiliser la commande suivante:

route add default gw 192.168.0.1 eth0

Pour supprimer une route utiliser la commande :

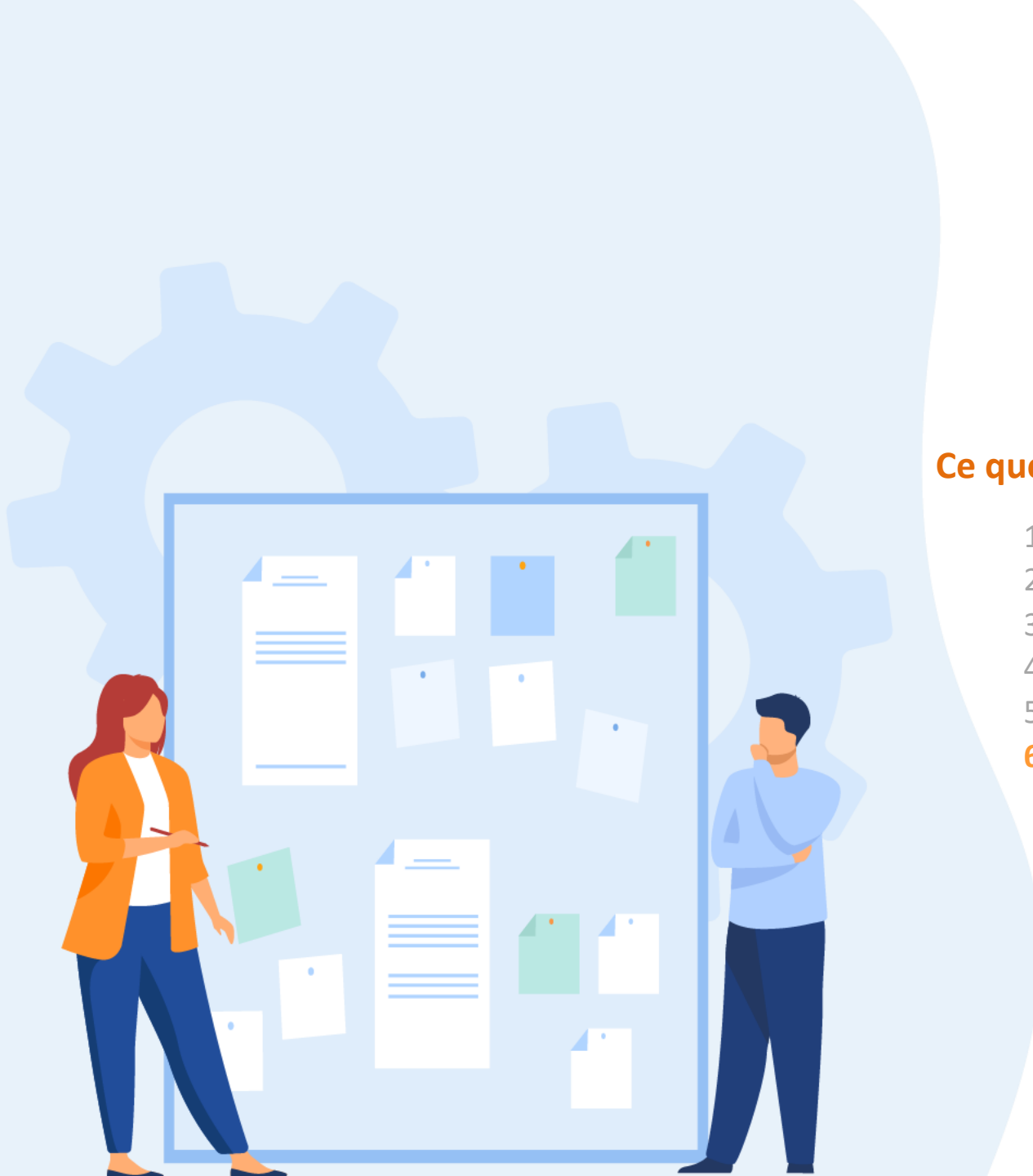
route del -net network-address -gateway gateway-address

CHAPITRE 2

MANIPULER Windows PowerShell & Linux Shell

Ce que vous allez apprendre dans ce chapitre :

1. Découvrir le « PowerShell »
2. Fonctionnement Windows PowerShell
3. Découvrir le « SHELL »
4. Les Commandes de base
5. les scripts shells
6. **Présenter les outils d'installation de package et applications**



02 – Présenter les outils d'installation de package et applications

RPM **abréviation (RedHat Package Manager)** est un gestionnaire de packages (logiciels) :

Il fournit les paquets et les outils de manipulation de ces paquets. Il est créé par RedHat en 1995 et distribué sous licence GNU GPL. RPM est un logiciel libre depuis 2019-2020.

Il est utilisé par de nombreuses distributions (Fedora, Mandriva, SuSe,...) et il permet l'installation, la suppression, la mise à jour, la vérification, des requêtes des programmes, très facilement et incluant la notion de dépendance. Voici les commandes **rpm** qui permettent l'installation de l'outil RPM, sa mise à jour et sa désinstallation.

- **Installation des rpm :**

`rpm -ivh coreutils-8.30-4.el8.x86_64.rpm`

- **Mettre à jour un paquet :**

`rpm -Uvh coreutils-8.30-4.el8.x86_64.rpm`

- **Désinstaller un paquet :**

`rpm -evh coreutils-8.30-4.el8.x86_64.rpm`

02 – Présenter les outils d’installation de package et applications

Voici la liste des commandes permettant la manipulation des RPMs qui sont illustrés dans la table ci-dessous.

COMMAND	TASK
rpm -qa	List all RPM packages currently installed
rpm -a NAME	Display the version of NAME installed on the system
rpm -qi NAME	Display detailed information about a package
rpm -q1 NAME	List all files included in a package
rpm -qc NAME	List configuration files included in a package
rpm -qd NAME	List documentation files included in a package
rpm -q --changelog NAME	Show a short summary of the reason for a new package release
rpm -q - - scripts NAME	Display the shell scripts run on package installation, upgrade, or removal

02 – Présenter les outils d'installation de package et applications

YUM est l'outil de gestion des paquets dans RedHat, Centos et Fedora (depuis Fedora22, yum remplacé par dnf). Voici les différentes commandes **yum**:

search : rechercher dans tous les paquets le terme openssh

yum search openssh

list : Recherche un paquet par son nom : yum list openssh

lister tous les paquets disponibles : **yum list**

info : Obtenir des infos sur le paquet httpd (serveur Apache http)

yum info httpd

install : Installe le paquet httpd et ses dépendances / faire des mise à jour yum

install httpd ; yum update

remove : Supprime le paquet openssh / paquet avec ses dépendances yum

remove openssh ; yum autoremove openssh

02 – Présenter les outils d’installation de package et applications

dpkg (pour Debian package) est l'outil de bas niveau gérant les paquets des distributions basées sur Debian.

Syntaxe de la commande dpkg : dpkg [options] [action] [package-files | package-name]. La liste des options est décrite dans la table ci-dessous.

Option	Forme longue	Description
-i	--install	Installer le paquetage
-r	--remove	Supprimer un paquet, mais laisse intacts les fichiers de configuration
-P	--purge	Supprimer un paquet, y compris les fichiers de configuration
-s	--status	Afficher des informations sur un paquet installé
-l	--info	Afficher des informations sur un paquetage désinstallé
-l	--list	Lister tous les paquets installés dont le nom correspond à un motif donné
-L	--listfiles	Lister les fichiers installés associés à un paquet
-S	--search	Localiser le(s) paquet(s) qui possèdent le(s) fichier (s) spécifiée par le motif
	--configure	Reconfigurer un paquet installé
	--get-selections	Afficher les paquetages actuellement installés

02 – Présenter les outils d'installation de package et applications

L'outil **apt-get** : ses commandes sont décrites dans la tables ci-dessous.

Commande	Description
install	Afficher des informations sur les paquets indiqués
update	Afficher les noms de tous les paquets installés sur le système
upgrade	Lister toutes les dépendances d'un paquet et la liste de tous les paquets possibles qui satisfont ces dépendances.
remove	Afficher une liste des paquets disponibles.
source	Obtention des codes sources du paquet donné
Clean	Supprimer tout le cache
Autoclean	Supprimer le cache des paquets périmés :

CHAPITRE 3

PARAMÉTRER LE DÉPLOIEMENT DE LINUX



1. Assurer la compression et l'archivage
2. Présenter les outils d'installation de package et applications
3. Paramétrer l'installation de Linux
4. Assurer la Configuration post-installation

02 – Paramétrer le déploiement de Linux

Assurer la compression et l'archivage

Dans cette partie, on va comprendre comment compresser et archiver des fichiers sous Linux. Il existe plusieurs formats d'archivage comme le format **zip** qui est le plus connu sous Windows, mais on peut l'utiliser sous Linux. On note aussi le format **rar**. Il existe également le **gzip** et le **bzip2** qui sont puissants mais ce sont des alternatives qui ne sont pas capable de compresser plusieurs fichiers à la fois. Il existe un outils appelé **tar**, qui est un outil d'archivage qui permet d'assembler les fichiers avant la compression.

Outils d'archivage

Tar signifie « tape archiver » et permet d'archiver et restaurer des fichiers et arborescences sur tout support.

Créer une archive d'une arborescence :

```
tar -cvf archive.tar /root/
```

```
tar -cvf home.tar /home/student/
```

c : créer une archive tar / -v : afficher le détail des opérations / -f : assembler l'archive dans un fichier

Lister le contenu d'une archive :

```
tar -tf archive.tar #tar -tvf archive.tar
```

Ajouter un fichier à l'archive déjà créée : tar -rvf archive.tar fich_supplementaire

Rechercher un fichier dans une archive : tar -tvf archive.tar fich_recherché

Extraire (restaurer) une arborescence à partir d'une archive : tar -xvf archive.tar

02 – Paramétrer le déploiement de Linux

Assurer la compression et l'archivage

Outils de compression:

Parmi les outils de compression les plus répandus :

gzip :

- c'est le plus connu et le plus utilisé.
- Résultat : nom.tar.gz

bzip2:

- il est un peu moins fréquemment utilisé.
- Résultat : nom.tar.bz2.

Archiver et compresser en même temps

Archiver et compresser en même temps avec tar : le programme tar peut appeler lui-même gzip ou bzip2.

Archiver et compresser en gzip

```
tar cvfz archive.tar.gz /root/
```

Décompresser le .gzip (Extraire)

```
tar xvfz archive.tar.gz
```

Archiver une arborescence avec compression bzip2

```
tar cvfj archive.tar.bz2 /root/
```

Décompresser le .bzip2 (Extraire)

```
tar xvfj archive.tar.bz2
```